

Структура проекта и реальные контуры системы

Подготовлено как печатный пакет: без цвета, без декоративного мусора, с упором на читаемость на бумаге.

Worker backend	Admin mini-app	Backend tests	Storage	Main entry
13867 LOC 101 files	4434 LOC 41 files	5190 LOC 38 test files	D1 + KV hybrid runtime	worker.js fetch + sche

Зачем этот PDF
1. Быстро понять, что в репозитории является ядром, а что боковыми приложениями. 2. Увидеть настоящую runtime-схему без старых предположений из README. 3. Разложить Worker, admin-app, D1 и KV по реальным зонам ответственности. 4. Зафиксировать архитектурные расхождения, которые мешают думать о проекте цельно.

Главные выводы
Это не один проект, а связка из нескольких приложений. Worker — главный системный orchestrator. Admin-app сейчас platform-oriented God Mode. Persistence уже D1-first, но KV всё ещё критичен. Есть drift между docs, worker SQL и admin schema.

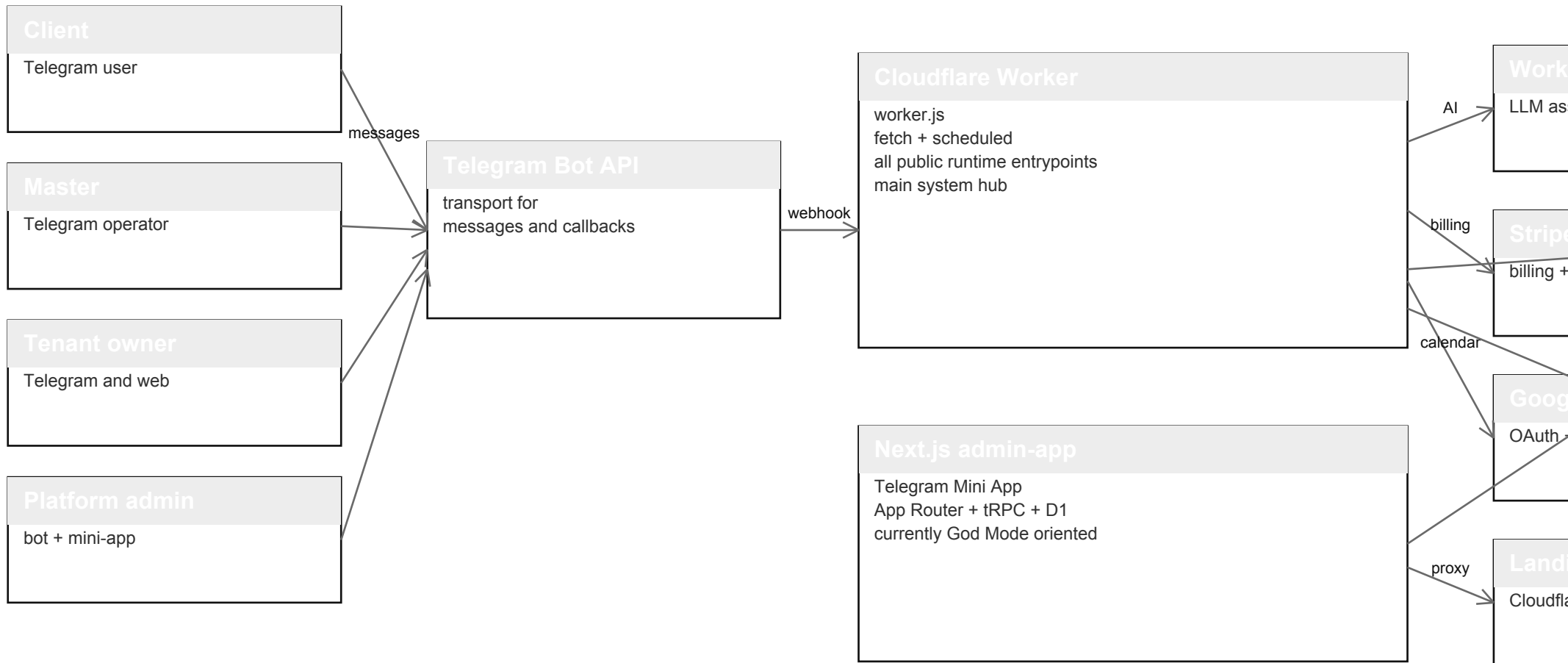
Ключевые точки входа

- manicbot/src/worker.js — webhook routing, Stripe, cron, HTML admin, Google OAuth callbacks, landing proxy.
- manicbot/src/handlers/message.js и callback.js — главный control flow пользовательских сценариев.
- manicbot/admin-app/src/app/api/trpc/[trpc]/route.ts — API boundary для mini-app.

manicbot/src/db/schema.sql и manicbot/admin-app/src/server/db/schema.ts — две competing схемы, которые сейчас расходятся.

System overview

actors, runtimes, storage and major boundaries



Interpretation

Система строится вокруг одного Worker-а: mini-app идёт параллельной дорожкой, а не прячется за ним. Persistence уже не KV-only: сейчас реальная модель — D1 + KV

Main packages

manicbot/src — Worker backend — 101 files / 13867 LOC
manicbot/test — backend tests — 39 files / 5190 LOC
manicbot/admin-app/src — mini-app — 41 files / 4434 LOC
manicbot-analysis/src — separate analysis UI — 47 files / 4339 LOC
manicbot-landing/src — marketing site — 7 files / 504 LOC

Worker folder breakdown

(root) — 7 files
admin — 2 files
billing — 6 files
handlers — 3 files
i18n — 57 files
roles — 1 files
services — 8 files
support — 1 files
tenant — 3 files
ui — 6 files
utils — 6 files

Backend hotspots by connectedness

manicbot/src/handlers/message.js — 27 internal imports
manicbot/src/handlers/callback.js — 25 internal imports
manicbot/src/worker.js — 19 internal imports
manicbot/src/ui/admin.js — 13 internal imports
manicbot/src/ai.js — 12 internal imports
manicbot/src/i18n/en/index.js — 12 internal imports
manicbot/src/i18n/pl/index.js — 12 internal imports
manicbot/src/i18n/ru/index.js — 12 internal imports

Admin-app hotspots by connectedness

manicbot/admin-app/src/server/api/root.ts — 10 internal imports
manicbot/admin-app/src/app/DashboardClient.tsx — 3 internal imports
manicbot/admin-app/src/app/api/trpc/[trpc]/route.ts — 3 internal imports
manicbot/admin-app/src/app/layout.tsx — 3 internal imports
manicbot/admin-app/src/server/api/trpc.ts — 3 internal imports
manicbot/admin-app/src/trpc/server.ts — 3 internal imports
manicbot/admin-app/src/app/agents/AgentsPageClient.tsx — 3 internal imports
manicbot/admin-app/src/app/appointments/AppointmentsPageClient.tsx — 3 internal imports

Worker internals

how manicbot/src is actually organized

Gateway / context

worker.js
tenant/resolver.js
tenant/storage.js
utils/db.js
wrangler routes + cron

Handlers

handlers/message.js
handlers/callback.js
handlers/cron.js
These files carry a lot of
real orchestration burden.

Domain services

services/appointments.js
services/users.js
services/services.js
services/state.js
services/chat.js
roles/roles.js
support/tickets.js

screens

lists

menus / prompts

UI and shared text

ui/screens.js, ui/admin.js, ui/sysadmin.js, ui/booking.js, ui/keyboards.js, i18n/*
UI is not an outer shell here: it is wired directly into flow control from handlers and AI actions.

Admin-app and persistence

what the web console is, and where data really lives

Admin-app actual role

Next.js App Router mini-app
TelegramGate hardcodes creator access
adminProcedure also checks creator / platform role
So this is platform God Mode, not tenant self-service yet

Admin-app stack

layout.tsx + Shell.tsx
app/api/trpc/[trpc]/route.ts
server/api/root.ts
server/api/routers/*
server/db/schema.ts

Important ac

Dashboard
Tenants
Users
Appointments
Agents
Billing / System /

D1: global/platform tables

tenants
bots
platform_roles
support_agents
tenant_support_agents
stripe_customers

D1: tenant-scoped business tables

appointments
users
masters
tenant_roles
services
tenant_config
blocked_users

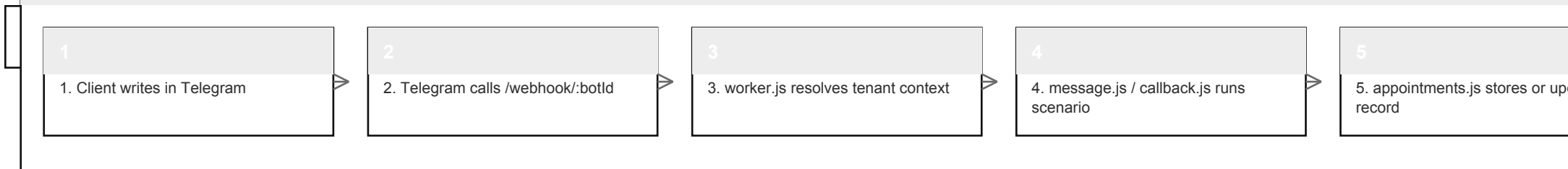
KV sidecar

bottoken:*
gcal:oauth:*
stripe:evt:*
tklock:*
legacy tenant

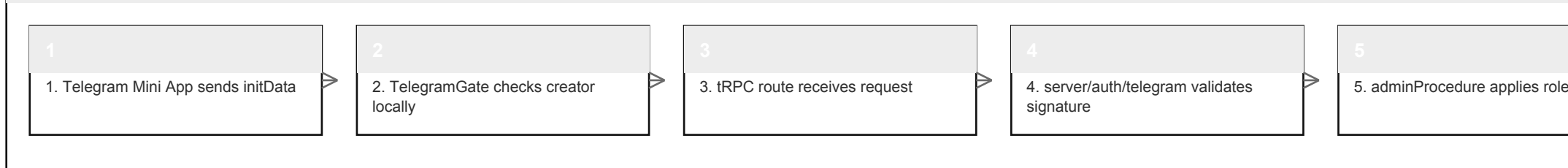
Storage reality

Система уже D1-first для сущностей, но KV остаётся обязательной operational прослойкой. Это не чистая D1-архитектура и не старый KV-only режим — это гибрид.

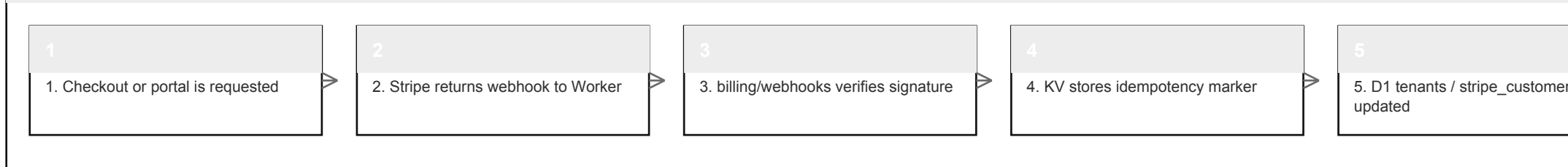
A. Booking from Telegram



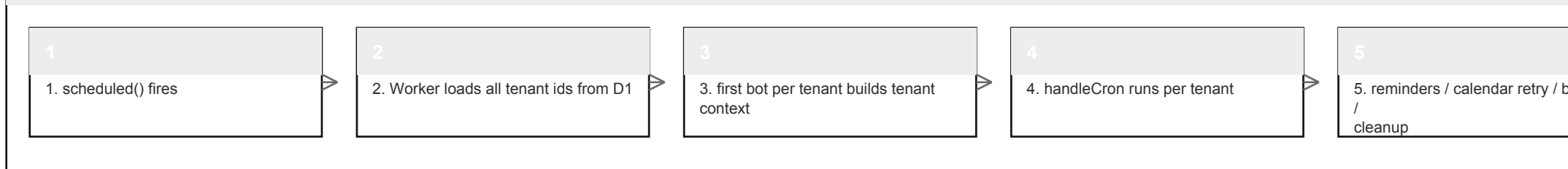
B. Mini-app request path



C. Billing lifecycle



D. Multi-tenant cron



Drift and cleanup order

where the project disagrees with itself

Docs drift

README still speaks KV-first.
MULTI_BOT_SETUP is outdated.
admin-app README is generic template.

Schema drift

worker SQL != admin schema
services ordering differs
ticket tables differ
masters fields differ

Role drift

Worker uses system_admin
tenant_owner / master
Mini-app still uses admin / owner
in some places

Most important architectural truth

There are two admin surfaces now: Worker HTML admin and Next.js God Mode. They are not two views over one clean backend surface; they are parallel operational tools.

Highest leverage cleanup order

1. Unify role vocabulary across Worker and admin-app
2. Unify billing status vocabulary and enum meanings.
3. Reconcile worker SQL schema with admin Drizzle s
4. Decide explicitly whether admin-app is platform-only
5. Update README and setup docs to the real hybrid

Print note

Этот PDF сделан именно для бумажного чтения: нормальная иерархия, чёрно-белая палитра и разбивка по страницам вместо одной огромной доски.